

Module 2

Generate and Save MHD Simulation Data

A short guide to creating trustworthy examples for the neural network

Module 2 has one job: run the reduced-MHD equations from Module 1 and save the numerical fields in a form that the neural network can use later.

Contents

1	The whole module in one picture	1
2	What one simulation contains	2
3	How the solver works	2
4	What is saved	3
5	How one simulation becomes learning examples	3
6	Keep complete simulations together	3
7	Checks before the data are accepted	4
8	What Module 2 does not claim	4
9	Completion checklist	5

1 The whole module in one picture

For each simulation, we choose the resistivity η , viscosity ν , and initial fields. The solver then calculates how the fields change with time and saves the result.



The saved arrays are the training data. A contour plot may help us look at a result, but the plot itself is not training data.

2 What one simulation contains

The solver uses the two equations established in Module 1:

$$\frac{\partial \psi}{\partial t} + [U, \psi] = \eta \nabla^2 \psi, \quad (2.1)$$

$$\frac{\partial \omega}{\partial t} + [U, \omega] = [J, \psi] + \nu \nabla^2 \omega, \quad (2.2)$$

with

$$J = -\nabla^2 \psi, \quad \omega = \nabla^2 U. \quad (2.3)$$

The bracket is simply the two-dimensional advection term:

$$[f, g] = f_x g_y - f_y g_x. \quad (2.4)$$

The version-1 example begins with

$$\psi_0(x, y) = \sin x \sin y, \quad U_0(x, y) = A_0(\cos x - \cos y). \quad (2.5)$$

Changing η , ν , or A_0 creates a different simulation case. The initial shape is fixed in version 1; only its flow amplitude changes through A_0 .

3 How the solver works

The domain is periodic in both x and y . The program represents each field with Fourier modes, calculates spatial derivatives, and advances the solution through small time steps.

The exact implementation may use Fourier transforms and a Runge–Kutta time step. Those details matter to the code, but the physical idea is simple:

1. Start with ψ_0 and U_0 .
2. Calculate J and ω from spatial derivatives.
3. Use the two MHD equations to calculate the time change.
4. Move forward by one small time step.
5. Repeat until the final time.

Very short wavelengths that cannot be represented reliably are removed before nonlinear terms are used. This prevents unresolved numerical patterns from appearing as false physics.

4 What is saved

One case file stores the numerical values needed to reproduce and interpret the simulation:

Saved item	Meaning
x, y, t	The grid and saved times
ψ, U	Magnetic-flux and flow fields
J, ω	Current and vorticity calculated from ψ and U
η, ν, A_0	The physical settings for this case
Initial conditions	The exact starting fields
Solver settings	Grid size, time step, and numerical method
Checks	Energy, peak current, and other useful histories

The preferred file type is HDF5. In plain language, HDF5 is one organized file that can hold large numerical arrays together with their labels and settings.

Saving the raw arrays preserves the physics. Saving only pictures would lose numerical precision, coordinates, units, and most of the information needed for training.

5 How one simulation becomes learning examples

The neural network receives the initial fields, the two transport coefficients, and a requested time. It predicts the fields at that time:

$$(\psi_0, U_0, \eta, \nu, t) \longrightarrow (\psi(t), U(t)). \quad (5.1)$$

A single simulation can provide examples at many saved times. For example,

$$(\psi_0, U_0, \eta, \nu, t_k) \longrightarrow (\psi_k, U_k). \quad (5.2)$$

These examples are closely related because they came from the same simulation. They must therefore remain in the same data group.

6 Keep complete simulations together

The cases are divided into three groups:

- **Training cases** teach the network.

- **Validation cases** help choose training settings.
- **Test cases** measure performance only after training is finished.

All saved times from one simulation stay in one group. If early times from a case were used for training and later times from that same case were used for testing, the test would be too easy. The network would already have seen almost the same physical evolution.

Only the training cases are used to calculate any scaling applied to the data. The same saved scaling is then applied to validation and test cases.

7 Checks before the data are accepted

A smooth-looking animation is not enough. Before a simulation is used for training, the following checks should pass:

1. **Known simple solution:** a case with a known decay rate is reproduced.
2. **Grid check:** using a finer spatial grid does not materially change the answer.
3. **Time-step check:** using a smaller time step does not materially change the answer.
4. **Periodic boundaries:** opposite edges match as required.
5. **Energy behavior:** resistivity and viscosity remove energy rather than create it.
6. **File check:** saving and reopening the case does not change the arrays or swap the x and y directions.

For this model, the energy should obey

$$\frac{dE}{dt} = -\eta \int J^2 dA - \nu \int \omega^2 dA \leq 0. \quad (7.1)$$

This is a useful test because a sign error or unstable calculation can cause the numerical energy to increase when it should decrease.

8 What Module 2 does not claim

Module 2 creates data from a small, two-dimensional reduced-MHD model. It does not run M3D-C1 and it does not show that the saved cases reproduce a real experiment.

The file layout is designed so that genuine M3D-C1 output could be connected later. That later step would still require explicit checks of geometry, units, field definitions, boundary conditions, and physical models.

9 Completion checklist

Module 2 is complete when:

- one command runs a case and writes the full numerical arrays;
- the solver passes the simple-solution, grid, time-step, boundary, and energy checks;
- the saved file can be reopened without changing the data;
- every case has a unique identifier and complete settings;
- training, validation, and test groups contain different complete simulations; and
- the project clearly states that these are reduced-MHD simulations, not M3D-C1 results.

Core idea: Module 2 produces trustworthy MHD examples. Later modules compare a neural-network prediction with the saved answer and ask whether enforcing the MHD equations improves that prediction.